

Metodología I

Unidad 2

Selección y Justificación de Modelos de Desarrollo

Nombre: Farias, Gustavo

Comisión: M2025-13

Matrícula: 101662

Repositorio GitHub:

https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/Metodologia_I

Sección A: Análisis del Modelo en Cascada

1. Por qué Cascada funciona para "MedControl"

Para el proyecto del marcapasos, el modelo en Cascada es, sin duda, la opción más lógica. Lo que define a este proyecto es que los requisitos no van a cambiar: están atados al hardware y a leyes de salud muy estrictas. En Cascada, al ser un proceso secuencial (primero requisitos, luego diseño, luego código, etc.), nos obliga a tener todo claro antes de empezar a programar. Esto es vital aquí porque necesitamos una documentación impecable para los auditores. No podemos estar "probando a ver qué pasa"; tenemos que demostrar que cada línea de código cumple con un requisito específico de seguridad. La predictibilidad de este modelo nos da la tranquilidad de que, si seguimos las fases al pie de la letra, el resultado final será estable y seguro, que es lo único que importa en un dispositivo médico.

2. Por qué Cascada sería un desastre para "SocialEats"

Usar Cascada para la app de los food trucks sería un error enorme. El problema principal es que los fundadores no saben bien qué quieren hasta que no lo ven. En Cascada, si definimos mal los requisitos al principio (que es muy probable acá), vamos a pasar meses desarrollando algo que quizás al usuario no le guste nada, y nos daremos cuenta recién al final, cuando ya gastamos toda la plata. Además, la UI/UX es clave para que la app sea "adictiva", y en Cascada la interfaz se diseña al principio y se ve al final. Sin feedback temprano de los usuarios, es casi seguro que lancemos un producto aburrido o confuso. La rigidez de este modelo no permite esos cambios rápidos que necesita una startup.

3. La paradoja de Winston W. Royce

Siempre se cita a Royce como el "padre" de la Cascada, pero en realidad él advirtió que el modelo puramente secuencial era riesgoso. La industria tomó su diagrama inicial y lo usó como regla de oro, ignorando que Royce proponía incluir iteraciones. Él sugería hacer, básicamente, dos pasadas: una primera vuelta para validar el diseño y los requisitos (como un prototipo) y recién después hacer el desarrollo final. O sea, él nunca quiso decir que hay que ir en línea recta sin mirar atrás; propuso agregar feedback entre las etapas para evitar llegar al final con un producto erróneo.

Sección B: El Modelo de Prototipado

1. Por qué Prototipado es ideal para "SocialEats"

Acá el prototipado encaja perfecto porque la clave es "ver para creer". Como los requisitos son ambiguos, la mejor forma de trabajar es construir versiones rápidas y descartables de la app para que los fundadores y los usuarios las prueben. Esto nos permite centrarnos en lo que más importa: la experiencia de usuario (UI/UX). Si una pantalla no es intuitiva, la cambiamos en el siguiente prototipo sin haber perdido meses programando la lógica de fondo. Es la mejor manera de reducir la incertidumbre y asegurarnos de que estamos construyendo algo que la gente realmente va a querer usar.

2. El riesgo del "Scope Creep" (Alcance no controlado)

Lo que tendríamos que cuidar mucho con los fundadores es que no se entusiasmen de más pidiendo funciones nuevas todo el tiempo. Como el prototipado es muy flexible, es fácil caer

en la trampa de decir "ya que estamos, agreguemos esto otro", y así el proyecto nunca termina o se vuelve impagable. Hay que dejar claro desde el principio que el objetivo del prototipo es aprender y validar, no tener la versión final con todas las campanas. Si no gestionamos bien esas expectativas, podemos terminar con un monstruo de funcionalidades que no sabemos cómo cerrar.

3. La idea de Brooks: "Plan to Throw One Away"

Lo que dice Brooks de "construir uno para tirar" tiene todo que ver con esto. Básicamente, nos dice que el primer intento siempre va a tener errores de concepto porque todavía no entendemos bien el problema. Entonces, en vez de tratar de que ese primer código sea perfecto, hacemos un prototipo rápido sabiendo que lo vamos a descartar. Ese "tiempo perdido" en realidad es una inversión: nos enseña qué no funciona. Así, cuando empecemos el desarrollo real, ya vamos a saber exactamente qué hacer, evitando errores caros a largo plazo. Es mejor perder tiempo ahora que rehacer todo después.

Sección C: Modelo Espiral y Conclusión

1. Los riesgos dominantes:

- En MedControl: El riesgo es técnico y de seguridad. Si el software falla, alguien puede morir. Es un riesgo catastrófico.
- En SocialEats: El riesgo es de mercado. El miedo es gastar plata en desarrollar una app que técnicamente funcione bien, pero que nadie quiera usar porque no les gusta o no la entienden.

2. ¿Podría usarse Espiral en MedControl?

Sí, el modelo Espiral de Boehm podría servir para MedControl porque su fuerte es la gestión de riesgos. En cada vuelta de la espiral, analizamos los riesgos más graves primero. Para un marcapasos, esto permitiría atacar los problemas de seguridad críticos desde el día uno. Sin embargo, es un modelo caro y complejo de administrar. Aunque es viable, quizás sea demasiado pesado si los requisitos ya son tan fijos, pero sin duda ofrecería un nivel de seguridad extra al evaluar constantemente los fallos potenciales.

3. Recomendación Final

Para el Proyecto A (MedControl):

Mi recomendación final es usar el Modelo en Cascada. Sé que suena antiguo, pero en este caso la estabilidad es reina. Como los requisitos no van a cambiar y necesitamos cumplir normas estrictas, la estructura rígida de la Cascada nos ayuda a mantener el orden y la documentación clara para los auditores. No necesitamos flexibilidad, necesitamos certeza. Cualquier desviación o cambio improvisado podría poner en riesgo vidas, por lo que el enfoque paso a paso, verificando cada fase antes de pasar a la siguiente, es la única forma responsable de garantizar que el firmware sea 100% fiable.

Para el Proyecto B (SocialEats):

Para la startup, la elección obvia es el Modelo de Prototipado. Acá el enemigo es la incertidumbre del mercado, no la complejidad técnica. Necesitamos sacar algo rápido para

ver si a la gente le gusta la interfaz y la idea. El prototipado nos permite fallar barato y rápido: si una idea no funciona, la tiramos y probamos otra sin haber invertido meses de desarrollo. Esto mantiene contentos a los inversores porque ven progreso tangible y nos asegura que, cuando lancemos la versión final, estemos ofreciendo algo que los usuarios realmente valoran y disfrutan.